VS Code Without Source Control

by Bryan A. *CrazyUncleBurton* Thompson

Last Updated 3/6/2026

Overview

Millions of hours are wasted every year installing and configuring unique IDEs and compilers and libraries every time the developer wants to try a new microcontroller platform.

VS Code is an Integrated Development Environment from Microsoft. Source Control is a systematic method of working with an existing project or as part of a team. We maintain backup copies of the project in GitHub as we develop, and multiple people can work on a project at the same time. But source control is confusing and beginners can choose to put off learning this and get on with uploading the project to their microcontrollers!

Our goal is to create a unified development environment with concurrent versioning which supports thousands of microcontrollers and development boards. VS Code and all the extensions and support apps mentioned in this document are all free downloads.

Prerequisites

1. Admin rights on the computer are needed to install some apps.

Setting Up a New Computer

Note: This section only needs to be done once for each computer.

1. **Browse** to https://code.visualstudio.com/download#
2. **Download VS Code for the OS the computer has installed**. For Windows, use the one marked System instead of the one marked User and instead of the Windows Store version. It will install for all users and give the ability to edit important files.
3. **Right-click the downloaded file and choose Run As Administrator**
4. Give the popup window permission **Yes**
5. **Accept** the license agreement.
6. Choose the default location to install VS Code and **click Next**
7. **Click the Create a Desktop Icon and "Open with Code" options** then **click Next** and then **click Install**
8. Leave Launch Visual Studio Code checked and then **click Finish**
9. Open VS Code.
10. From the left-hand toolbar, **click the Extensions icon**:
11. Search **Platform IO** and **install the Platform IO extension**
12. **Trust Platform IO** if a popup box appears.
13. **Click Reload Extensions** after Platform IO installs.
14. From the left-hand toolbar, **click the Extensions icon**:
15. Search for **C/C++**
16. **Install the Microsoft C/C++ Extension** if it's not already installed. It's Microsoft, so no trust needed.
17. **Restart VS Code**

The Platform IO Project

The Platform IO project contains files and folders that with the source code which gets compiled and uploaded to the microcontroller. It also contains files like platformio.ini which tell Platform IO and the compiler what microcontroller we are compiling for. The project can contain one branch or multiple branches for things like past, production and development environments.

When we open the project in VS Code, we are copying the contents of the GitHub repository to the local hard drive. If there is no existing project for us to copy, we show the process of creating a new project.

The quickest way to get started in microcontroller development is to find an existing project that works on the microcontroller we're using, and to copy it to the local machine. This project might be in the microcontroller manufacturer's GitHub account (common), or it might be a project by some rando on the Interweb.

Project Structure

.pio/
  build/ - temporary build files, clean to delete these
  libdeps/ - managed (dynamic, changing) libraries go here

.vscode

platformio.ini
README.md – a markdown-style file that creates the docs in your GitHub project.

data/ - Stuff that is stored in the SPIFFS/LittleFS file systems in microcontroller memory.
  image_1.png
  image_2.png

lib/ - These are unmanaged (static) libraries
  unmanaged_library_1/
    unmanaged_library_1.h
    unmanaged_library_1.c

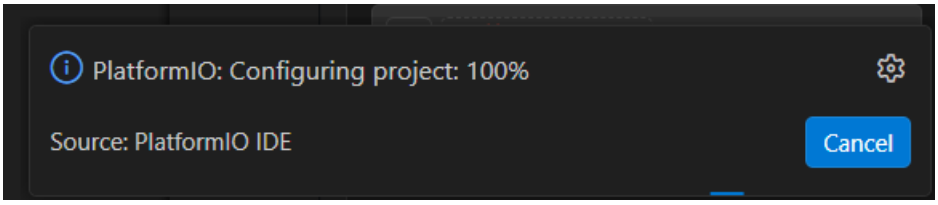
include/
  pin_config.h – a good place to put mapping of variable names to specific pins on the microcontroller. Sometimes supplied by the manufacturer.

src/
  main.cpp – Arduino /ino style file which contains the starting point for your source, or maybe all of it.

Opening an Existing Project in VS Code

Note: If there is no existing project to clone, skip ahead to the section marked **Create a New Platform IO Project in VS Code**

1. Open VS Code and close all folders and workspaces and projects in VS Code. **File/Close Folder** or **File/Close Workspace** will accomplish this.
2. **Browse** to the GitHub project you want to clone.
3. **Click the green Code button** and then click Download Zip.
4. Extract the contents of the Zip and place that project folder in this folder (create the parent folder if needed): **UserProfile\Documents\PlatformIO\Projects**
5. Go to VS Code and click **File / Open Folder**
6. **Select the project folder you just extracted.**
7. Platform IO will download the managed libraries and install a compiler for this CPU type. This may take some time, and looks like this:



8. Wait for all tasks to complete before proceeding!

Create a New Platform IO Project in VS Code

Note: If you cloned an existing project earlier, you do not need to create a new project. Skip ahead to the section **Compiling and Uploading to the Microcontroller**

1. **Click the alien head Platform IO logo** from the left-hand toolbar.
2. From the Quick Access menu (bottom left), choose **Projects & Configuration**
   Click the **+ Create New Project** button
3. **Choose a name**. Choose anything. This is temporary. The project name will be chosen later.
4. **Choose the development board** from the list which most closely matches the one we're working with.
5. Choose the Framework **Arduino**
6. Check the box that says **Location: Use Default Location** This directory is **User Documents\PlatformIO**
7. Click the **Finish** button and let it build the project.
8. Close the temporary project by choosing **File/Close Folder**
9. Browse to **User Documents\PlatformIO** and in that folder create the folder **Projects**
10. Create a folder in Projects to hold all of the files and folders associated with this project.
11. We should see the folder structure of our project, along with the platformio.ini file that tells Platform IO how to compile our project.
12. Go back to VS Code**. Click the Explorer icon** at the top left of the left toolbar. This shows us the files in the local project.
13. If you have sample code from the manufacturer of the microcontroller or sensor(s), **copy that code to the project.** Otherwise **Create a file** called **/src/main.cpp** It should be of the same structure as an Arduino .ino file, with a setup function that runs once and a loop function that runs forever.
14. Create any other files that your project needs.
15. If libraries (shared code) are supplied by the microcontroller or the devices the controller is connecting to, **click the Platform IO icon** in the left toolbar and select **Libraries** from the Quick Access pane.
16. Select the library that supports your hardware and click it to browse to that library.
17. Click the **Add to Project** button
18. Select our project from the list and click "Add" to download it to the location of the managed libraries: **/.pio/libdeps**
19. If a library is not available via Arduino Platform IO download, you can manually add that library to the unmanaged libraries folder **/lib**

Compiling and Uploading to the Microcontroller

Now that we have a project,

1. Connect a USB cable from the computer that VS Code is running on to the microcontroller.
2. In VS Code, **click the Platform IO Upload icon** in the bottom toolbar. The hover text will be Platform IO: Upload.
3. This takes the C source code and compiles it into machine code for the specific microcontroller configured in the project. Then VS Code will transfer that machine code to the microcontroller – this is called uploading.
4. When it's ready to upload, we see lines like "Connecting…" and "Writing…". Wait for the line that says SUCCESS in green text before proceeding:

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS
* Executing task: C:\Users\batee.DESKTOP-OFFICE\.platformio\penv\Scripts\platformio.exe run
Advanced Memory Usage is available via "PlatformIO Home > Project Inspect"
RAM:   [=         ]   9.0% (used 29516 bytes from 327680 bytes)
Flash: [=         ]   7.1% (used 467420 bytes from 6553600 bytes)
Building .pio\build\lilygo-t-display-s3\firmware.bin
esptool.py v4.8.5
Creating esp32s3 image...
Merged 2 ELF sections
Successfully created esp32s3 image.
============================================= [SUCCESS] Took 25.69 seconds ====================
* Terminal will be reused by tasks, press any key to close it.

Default (OtterPilot-v3.8.0-Plan-B)    Auto
```

5. When finished, the microcontroller will reboot and start running the program uploaded to it.

Scrapping Changes and Starting Over

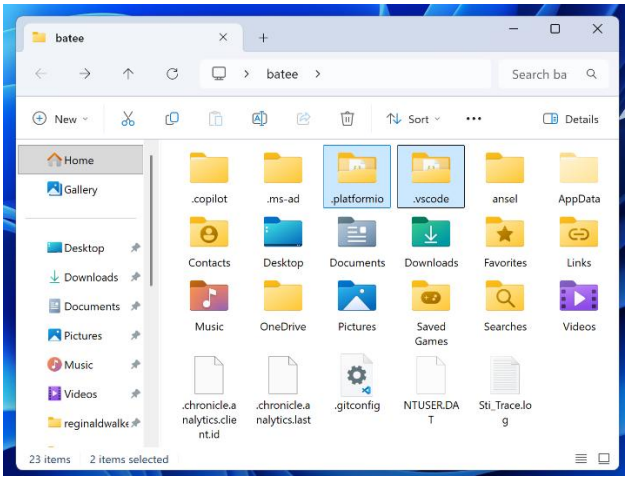
If things aren't working out, you can always start over:

1. Close VS Code folder or workspace with **File/Close Folder** or **File/Close Workspace**
2. Go to **User Documents\PlatformIO\Projects**
3. Rename the folder so that we can re-download the folder from the project. It will have the name of the repository. Or if it really is no longer needed, just delete it
4. Go to the section marked **Opening an Existing Project in VS Code** or **Create a New PlatformIO Project in VS Code** and start again.

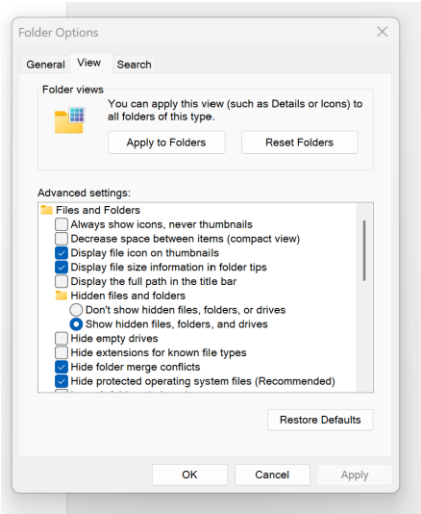
Things Still Aren't Working

It happens. The changes we're making with AI can mess something up in the VS Code installation, causing us to have to reinstall VS Code and its extensions.

1. Close VS Code.
2. Go to the Start Menu and **click Settings** and then **click Apps** and then **Installed Apps**.
3. Find Microsoft Visual Studio Code in the list and uninstall it.
4. Once the uninstaller completes, go to the **User\Documents** folder:

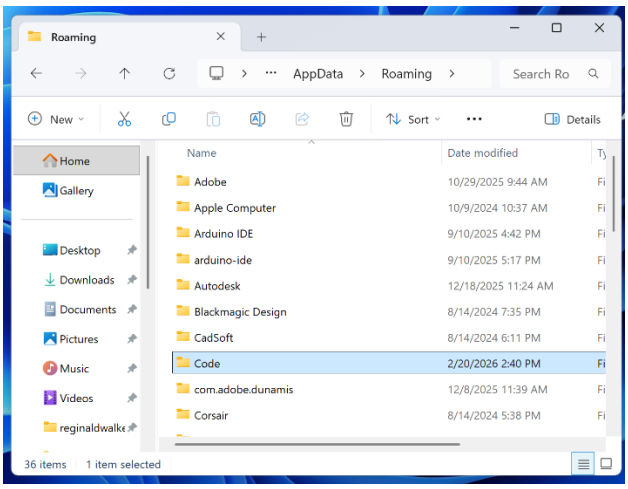


5. **Highlight the folders .platformio** and **.vscode**. If they don't appear in the list of folders, it may be necessary to click "… and then Options and then View to see Folder Options:



6. **Check Hidden files, folders, or drives** and then **click OK**
7. Delete the folders marked **.platformio** and **.vscode**.

8. Browse into the **UserID\App Data** folder, and then the Roaming folder:



9. **Highlight the folder Code** and **delete it**
10. Reboot Windows to clear any pending file and folder deletions.
11. Go to the section marked **Setting Up a New Computer** and follow the instructions to reinstall and reconfigure VS Code.